Summary

Modern scientific research requires consistent document preparation, reproducible computation, unified data management, and transparent figure provenance—yet researchers rely on fragmented tools that increase cognitive load and introduce irreproducibility. We present SciTeX, an AI-native, modular research automation platform providing four integrated modules: Writer for structured LaTeX manuscript preparation with containerized builds; Scholar for literature management with automated metadata extraction; Viz for publication-quality figure generation with embedded provenance; and Code for reproducible computation with GPU and container support. Each module operates independently while sharing a unified project structure through lightweight symlinks and global identifiers, enabling cross-module traceability. SciTeX reduces the activation energy required for rigorous, reproducible science while maintaining compatibility with traditional workflows. The platform is open source and follows FAIR principles. This manuscript demonstrates SciTeX's self-descriptive capabilities by being prepared within the platform it documents.

Highlights

- SciTeX integrates manuscript writing, figure generation, literature management, and computation in a unified platform
- Modular architecture enables independent use of each component while providing synergy through integration
- Containerized compilation ensures reproducible manuscript output across all operating systems
- Embedded metadata enables full provenance tracking from figures and results back to generating code
- AI-native design provides integration points for LLM-assisted research workflows

SciTeX: An AI-Native Modular Platform for Reproducible Scientific Research

Yusuke Watanabe[a,*]

[a] *SciTeX.ai, Tokyo, Japan*

Keywords: reproducible research, scientific workflow automation, data science platform, AI-native tools, figure provenance, FAIR principles

˘ 8 figures, 3 tables, 136 words for abstract, and 3056 words for main text

1. Introduction

Scientific workflows increasingly span multiple heterogeneous environments: personal laptops, HPC systems, cloud servers, and containerized pipelines [1, 2]. Traditional manuscript preparation tools remain fragmented, requiring manual management of figures, references, code execution, and version control. These inconsistencies lead to irreproducible results, duplicated effort, and high cognitive load that distracts researchers from their primary scientific objectives [3].

The preparation of scientific manuscripts involves numerous technical challenges that extend beyond the intellectual task of communicating research findings. Researchers must navigate complex typesetting systems, manage multiple document versions, coordinate figures and tables across formats, and ensure reproducible compilation environments [4]. Furthermore, maintaining consistency between computational analyses, their visual representations, and the manuscript text requires constant manual synchronization—a process prone to error and version drift [5].

Traditional approaches to manuscript preparation typically rely on local LaTeX installations, where specific versions of packages and compilation

*Corresponding author. Email: ywatanabe@scitex.ai

tools can vary significantly across machines and over time. This variability creates reproducibility challenges, particularly in collaborative environments where multiple authors work on different systems [6, 7]. Similarly, figure generation often occurs in isolation from the manuscript, with researchers using various plotting libraries without standardized metadata or provenance tracking. Literature management frequently depends on external tools that operate independently from the writing environment, leading to citation inconsistencies and manual bibliography maintenance.

Existing solutions have addressed individual aspects of this problem. Cloud-based platforms such as Overleaf provide consistent compilation environments but require continuous internet connectivity and separate figure generation workflows. Zotero and Mendeley excel at citation management but lack integration with manuscript preparation and computational analysis. Jupyter notebooks and Binder enable reproducible computation but provide limited support for publication-quality figures or structured manuscript writing [8]. Manubot pioneered open collaborative writing with automated citation processing [?], while Pandoc Scholar demonstrated agile multi-format document generation [9]. SigmaPlot and Origin offer sophisticated visualization capabilities but operate as standalone applications without provenance tracking or manuscript integration.

The fundamental challenge lies in unifying these disparate tools while maintaining the flexibility that researchers require [10]. The solution must accommodate diverse content types, multiple output documents, and varying journal requirements while ensuring a single source of truth for shared elements. Additionally, modern research increasingly involves AI-assisted workflows for code generation, literature review, and manuscript revision—capabilities that require native integration rather than ad-hoc addition [11, 12].

Version control systems, particularly Git, have emerged as powerful tools for managing research workflows [13, 14], enabling transparent tracking of changes and facilitating collaboration [15, 16]. However, effectively combining

Git with LaTeX, figure generation, and literature management requires significant technical expertise that many researchers lack [17].

Containerization technologies such as Docker have transformed software reproducibility [6, 18, 19], and several frameworks have emerged to leverage containers for reproducible research [20, 21]. The Rockerverse ecosystem demonstrates how containers can support reproducible R-based workflows [18], while tools like CREDO simplify Docker configuration for bioinformatics [22]. End-to-end templates for reproducible analysis and manuscript writing have been proposed [23, 24], yet comprehensive integration across all research activities remains elusive.

SciTeX addresses these challenges through a unified, modular, AI-native platform that allows researchers to: write manuscripts in a structured LaTeX environment with containerized compilation; manage citations and PDFs with consistency and automated metadata extraction; generate publication-quality figures with embedded provenance metadata; and execute analyses in controlled environments with synchronized results [25]. Importantly, SciTeX maintains bidirectional low migration cost: each module functions independently, providing value without requiring adoption of the entire ecosystem, while synergy across modules enables a strictly superior integrated workflow.

The platform follows principles articulated in seminal works on reproducible research [3, 1, 26] while incorporating modern infrastructure for containerized compilation [27, 28], workflow automation [29, 30], and FAIR data practices [31]. By embedding reproducibility infrastructure into everyday research tools, SciTeX makes rigorous practices the path of least resistance rather than an additional burden [32, 33].

Many researchers lack unified tools to integrate writing, computation, visualization, and literature management within a single, reproducible environment. Graduate students and early-career researchers often spend disproportionate time learning fragmented toolchains rather than conducting research. Computational scientists require reproducible pipelines that connect analysis code to manuscript figures without manual intervention [34].

AI-augmented research workflows demand native integration points for automated assistance. SciTeX provides this missing infrastructure, reducing the activation energy required for rigorous, reproducible science while democratizing access to advanced computational workflows for researchers unfamiliar with Linux, Git, or containers.

This manuscript is prepared within SciTeX Writer, referencing figures generated in SciTeX Viz, bibliography managed in SciTeX Scholar, and computational environments controlled by SciTeX Code—thereby making the manuscript a live demonstration of the platform's integrated capabilities.

2. Methods

2.1. System Architecture

SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and a Python package (v2.4.1+) installable via `pip install scitex`. The architecture employs lazy module loading via a custom `_LazyModule` wrapper, enabling fast imports while providing access to over 50 specialized modules. This modular design follows best practices for scientific Python applications [29, 1]:

```
import scitex as stx
stx.plt       # Publication plotting
stx.io        # Universal file I/O (30+ formats)
stx.scholar   # Literature management
stx.session   # Experiment lifecycle
stx.vis       # Visual figure specifications
stx.writer    # Manuscript preparation
```

Each module operates independently while sharing a unified project structure through lightweight symlinks. Global identifiers (SHA256 hashes) ensure traceability between figures, code, bibliography entries, and manuscript content, enabling reverse lookup capabilities (e.g., "which script generated this figure?"). This approach addresses concerns raised in discussions of figure provenance and data reproducibility [5, 35].

2.2. Session Decorator for Reproducible Experiments

The `@stx.session` decorator provides automatic experiment lifecycle management, implementing principles from reproducible research frameworks [3, 33]:

```
import scitex as stx
%DIF >
@stx.session
def analyze(data_path: str, threshold: float = 0.5):
    '''Analyze data file.'''
    data = stx.io.load(data_path)
    result = process(data, threshold)
    stx.io.save(result, "output.csv")
    return 0
%DIF >
if __name__ == '__main__':
    analyze()  # CLI: python script.py --data-path x --threshold 0.7
```

The decorator automatically: (1) generates CLI argument parsers from function signatures with type hints, (2) initializes session directories with timestamped run IDs, (3) injects global variables (`CONFIG`, `plt`, `COLORS`, `rng_manager`, `logger`), (4) manages random state for reproducibility, and (5) handles cleanup and optional notifications on completion. This design is informed by the "good enough practices" paradigm [1] and project management tools for team science [34].

2.3.

2.3. Publication-Quality Plotting (stx.plt)

The `stx.plt` module wraps matplotlib with automatic configuration from `SCITEX_STYLE.yaml`:

```
import scitex.plt as splt
%DIF >
fig, ax = splt.subplots(axes_width_mm=40, axes_height_mm=28)
ax.plot([1, 2, 3], [1, 4, 9])
ax.set_xyt("Time", "Value", "Analysis Results")
stx.io.save(fig, "figure.png")  # Exports PNG + JSON + CSV
```

Key features include: (1) automatic style application on import (font sizes, line widths, spine visibility), (2) `FigWrapper` and `AxisWrapper` classes providing `set_xyt()` for simultaneous x-label, y-label, and title setting, (3) automatic CSV export of underlying plot data alongside figures, (4) JSON sidecar metadata encoding axis bounds, tick locations, panel geometry, and color palettes, and (5) `stx.plt.load()` for figure reconstruction from saved metadata. This triple-export approach (image, metadata, data) addresses the reproducibility challenges identified in computational figure generation [5, 4].

Style values can be customized via YAML configuration, environment variables (`SCITEX_PLT_FONTS_AXIS_LABEL_PT=8`), or direct function parameters.

2.4. Universal File I/O (stx.io)

The `stx.io` module provides format-agnostic file operations supporting over 30 formats, following the principle of unified interfaces for data management [32]:

```
import scitex as stx
%DIF >
# Automatic format detection
data = stx.io.load("data.csv")       # DataFrame
config = stx.io.load("config.yaml") # Dict
model = stx.io.load("model.pkl")     # Python object
arrays = stx.io.load("data.h5")      # HDF5 exploration
```

```
%DIF >
# Unified save interface
stx.io.save(df, "output.csv")
stx.io.save(fig, "figure.png")      # PNG + JSON + CSV
stx.io.save(config, "config.yaml")
```

Supported formats include: CSV, JSON, YAML, Pickle, HDF5, Zarr, NumPy arrays, PyTorch tensors, images (PNG, JPG, SVG, PDF), audio, video, and MATLAB files. The module includes `H5Explorer` and `ZarrExplorer` for interactive hierarchical data navigation, plus configurable caching for frequently accessed files.

2.5. Literature Management (stx.scholar)

The Scholar module provides multi-source literature search and management, integrating capabilities typically found in standalone reference managers:

```
from scitex.scholar import Scholar
%DIF >
scholar = Scholar()
papers = scholar.search("deep learning", year_min=2022)
papers.save("bibliography.bib")
```

The module integrates with multiple metadata engines (CrossRef, Semantic Scholar, OpenAlex) via the `ScholarEngine` interface. Features include: automatic DOI detection and metadata extraction from PDFs, `ScholarPDFDownloader` for paper acquisition, `ScholarLibrary` for local storage management, and `Paper`/`Papers` classes with filtering and export capabilities. This design complements tools like Manubot [**?**] by providing direct integration with manuscript preparation.

2.6. Cloud Platform Architecture

The Django 5.1-based cloud platform provides web interfaces for all modules, following principles of continuous integration for scientific publishing [36]:

Code App: Browser-based Python execution with real-time output streaming, file upload/download, and workspace management.

Viz App: Canvas-based figure editor using Fabric.js 5.5.2 for interactive element manipulation, with JSON specification import/export for programmatic control.

Writer App: Structured LaTeX manuscript preparation with containerized compilation (Docker/Apptainer), section-separated authoring, and automatic figure format conversion.

Scholar App: Literature cards with abstract preview, PDF access, metadata editing, and direct citation insertion into Writer documents.

The platform employs TypeScript for frontend logic, PostgreSQL for data persistence, and Docker for containerized compilation ensuring consistent output across environments.

2.7. Containerized Compilation

Writer module compilation leverages containerization for reproducibility, implementing guidelines for Docker-based scientific workflows [6, 7]:

Multi-Engine Support: Tectonic [27] for ultra-fast incremental builds (1–3 seconds), latexmk for reliable industry-standard compilation (3–6 seconds), and traditional 3-pass compilation for maximum compatibility.

Environment Isolation: Docker and Apptainer/Singularity containers encapsulate specific TeX Live versions and all required packages, eliminating host system dependencies and guaranteeing identical PDF output across macOS, Linux, Windows, and HPC environments [19, 18].

Version Tracking: Git integration [13] with automatic latexdiff generation facilitates peer review processes [37].

2.8. AI-Native Workflow Integration

SciTeX provides native integration points for AI-assisted workflows, anticipating the growing role of AI in scientific research [11, 12]:

Structured APIs: JSON-based figure specifications, typed function signatures for CLI generation, and metadata formats amenable to automated analysis enable LLM-assisted code generation and figure improvement.

Workflow Hooks:

The session decorator and modular architecture provide hooks for AI-assisted revision, automated citation recommendation, and semantic search across analyses and literature.

Research Co-pilot Vision:

The architecture supports future development of automated analysis pipelines, figure generation, and manuscript drafting while maintaining researcher oversight and scientific integrity [38].

2.9. Implementation Details

The system supports deployment on local machines, cloud servers (AWS, GCP, Azure), and self-hosted NAS environments. Key technical specifications:

- Python 3.12+ with GPU acceleration support (CUDA 11.x/12.x)
- Django 5.1 with PostgreSQL backend
- TypeScript frontend with webpack bundling
- Docker/Apptainer containerization for reproducible execution [6]
- SLURM integration under active development for HPC workflows

The platform follows FAIR principles [31] and aligns with the Turing Way guidelines for reproducible research [26].

3. Results

This section presents validation results and demonstrations rather than experimental findings, consistent with software paper conventions. SciTeX is validated through real-world deployment and systematic testing across multiple research scenarios.

3.1. Session Decorator Validation

The `@stx.session` decorator successfully automates experiment lifecycle management, implementing principles from reproducible research guidelines [3, 1]. Testing confirms:

- Automatic CLI generation from function signatures with type hints correctly parses arguments and provides help text
- Session directories are created with timestamped IDs (e.g., `2025Y-11M-18D-07h53m37s_Z5MR`) enabling experiment organization
- Injected globals (`CONFIG`, `plt`, `COLORS`, `rng_manager`, `logger`) are accessible within decorated functions
- Random state management via `rng_manager` ensures reproducible results across runs
- Session cleanup handles errors gracefully while preserving outputs

The decorator transforms research scripts into production-ready CLI tools without requiring manual argparse configuration, reducing boilerplate code by approximately 50–100 lines per script. This approach aligns with the "good enough practices" paradigm for scientific computing [1].

3.2. Publication Plotting (stx.plt)

The plotting module achieves publication-quality output with minimal configuration:

-
- Automatic style configuration on import applies 300 DPI, Arial fonts, and consistent line widths
- `FigWrapper` and `AxisWrapper` classes provide enhanced methods (`set_xyt()`, `set_meta()`)
- Style values configurable via YAML, environment variables, or function parameters
- Figure dimensions specified in millimeters for journal compliance
- Automatic margin calculation from axes dimensions

Critically, `stx.io.save(fig, "figure.png")` automatically exports three files: PNG image, JSON metadata (axis bounds, tick locations, color palettes), and CSV data (underlying plot values). This triple export enables figure reconstruction via `stx.plt.load()`, addressing the common problem of irreproducible figures in scientific publications [5, 4].

3.3. Universal File I/O

The `stx.io` module demonstrates format-agnostic file operations following principles of unified data management [32]:

- Automatic format detection based on file extension handles 30+ formats
- `H5Explorer` and `ZarrExplorer` enable interactive navigation of hierarchical datasets
- Configurable caching reduces repeated file access overhead

- Unified `save()`/`load()` interface eliminates format-specific import statements

Testing confirms correct handling of CSV, JSON, YAML, Pickle, HDF5, Zarr, NumPy arrays, PyTorch tensors, and image formats across platforms.

3.4. Literature Management

The Scholar module provides multi-source literature search, complementing existing tools [?] with direct manuscript integration:

- `ScholarEngine` interface integrates CrossRef, Semantic Scholar, and OpenAlex
- `Paper` and `Papers` classes enable filtering by year, journal, citations
- `ScholarPDFDownloader` manages paper acquisition with rate limiting
- `ScholarLibrary` organizes local PDF storage with DOI indexing
- Bibliography export supports BibTeX and other formats

Integration with the Writer module ensures citation consistency: references added in Scholar automatically synchronize to manuscript `.bib` files.

3.5. Containerized Compilation

Writer module compilation achieves complete cross-platform reproducibility, implementing containerization best practices [6, 19]:

- Testing across Linux (Ubuntu, Debian, CentOS), macOS, and Windows Subsystem for Linux confirmed byte-for-byte identical PDF outputs using the same container image
- Multi-engine support: Tectonic [27] (1–3s), latexmk (3–6s), traditional 3-pass (12–18s)

- Apptainer/Singularity containers enable HPC environments without Docker [18]
- Automatic figure format conversion handles PNG, SVG, and PDF inputs

3.6.

The elimination of environment-dependent compilation issues represents a significant improvement over traditional local LaTeX installations, consistent with findings on containerized workflows [7, 22].

3.6. Cloud Platform Functionality

The Django-based cloud platform demonstrates effective web interfaces following continuous integration principles [36]:

Code App: Browser-based Python execution with real-time stdout/stderr streaming, workspace file management, and session persistence.

Viz App: Canvas-based figure editing using Fabric.js enables interactive element manipulation. JSON specification import/export bridges programmatic and visual workflows.

Writer App: Section-separated LaTeX editing with live preview, containerized compilation, and Git integration [13] for version control.

Scholar App: Literature cards display abstract, PDF access, and metadata editing. Drag-and-drop citation insertion into Writer documents.

3.7. Cross-Module Integration

The symlink-based architecture demonstrates effective module coupling, addressing integration challenges identified in reproducibility research [10, 23]:

- Figures generated in Code appear in Writer via symlink without file duplication
- Bibliography managed in Scholar synchronizes to Writer via shared `.bib` file

-
- SHA256 hashes enable queries across modules (e.g., “

  3.8.

-
-
- which script generated this figure?”)
- JSON metadata format enables automated analysis and AI-assisted workflows [11]

3.8. Self-Descriptive Validation

This manuscript validates SciTeX capabilities through direct demonstration, following the principle that reproducibility tools should be self-documenting [33, 24]:

- Authored in SciTeX Writer with section-separated LaTeX files
- Compiled in containerized environment ensuring cross-platform reproducibility
- Bibliography managed through SciTeX Scholar
- Modular structure demonstrates the framework’s organization principles

The manuscript is prepared within the platform it documents, ensuring accurate representation of system capabilities.

4. Discussion

SciTeX addresses fundamental challenges in modern scientific research by unifying manuscript preparation, figure generation, literature management, and reproducible computation into a cohesive, AI-native ecosystem [25]. The platform reflects principles derived from practical research frustrations: fragmented workflows, difficulty maintaining figure provenance [5], unreliable manual versioning, and high cognitive cost of switching between tools [1].

4.1. Design Philosophy and Modularity

The modular architecture enables researchers to adopt individual components incrementally while benefiting from increasing integration as more modules are utilized. This design philosophy—bidirectional low migration cost—ensures that each module provides standalone value. A researcher using only Writer gains containerized reproducibility without committing to the full ecosystem [6]. Adding Scholar provides integrated citation management. Incorporating Viz enables provenance-tracked figures. Finally, Code completes the pipeline with reproducible computation [3]. At each stage, the researcher gains capability without sacrificing flexibility or prior investments.

SciTeX democratizes advanced computational workflows—particularly for researchers unfamiliar with Linux, Git, or containers—without compromising power or flexibility. Graduate students beginning their research careers can leverage containerized reproducibility without understanding Docker internals [19]. Researchers transitioning from GUI-based tools like SigmaPlot can use familiar visual interfaces while gaining programmatic control. Those accustomed to Zotero maintain familiar literature management workflows while gaining manuscript integration.

4.2. Comparison with Existing Tools

SciTeX occupies a unique position in the research tooling landscape, integrating capabilities that existing solutions provide in isolation [10]:

Overleaf excels at collaborative LaTeX editing with consistent compilation but lacks integrated figure generation, computation, or literature management. SciTeX Writer provides comparable manuscript preparation with additional provenance tracking and local execution capability.

Zotero and Mendeley provide sophisticated citation management but operate independently from manuscript preparation and computational analysis. SciTeX Scholar offers comparable citation capabilities with direct manuscript integration.

Jupyter notebooks and Binder [8] enable reproducible computation and analysis but provide limited support for publication-quality figures or structured manuscript writing. SciTeX Code complements computational analysis with figure generation and manuscript integration.

Manubot [?] pioneered open collaborative writing with automated citation processing, demonstrating the value of Git-based manuscript workflows. SciTeX extends this approach with containerized compilation, integrated figure generation, and multi-module traceability.

SigmaPlot and Origin offer sophisticated visualization with GUI-based interfaces but lack provenance tracking, manuscript integration, or computational reproducibility. SciTeX Viz provides comparable visual capabilities with embedded metadata and cross-module traceability.

R-based workflows including rrtools [39], the Rockerverse [18], and repro [33] provide excellent reproducibility infrastructure for R users. SciTeX offers comparable functionality for Python-centric workflows while maintaining language flexibility.

The key differentiator is integration: SciTeX provides a unified platform spanning all components while maintaining independence of each module. This is not about replacing existing tools but about providing a coherent alternative for researchers who want an integrated workflow.

4.3. AI-Native Design

SciTeX is designed for an era where AI assistants are integral to research workflows [11, 12]. Rather than treating AI as an afterthought, the architec-

ture provides native integration points: structured APIs for code generation, metadata formats amenable to automated analysis, and workflow hooks for AI-assisted revision. This design enables researchers to collaborate effectively with LLMs and agents while maintaining scientific control [38].

4.4. Implications for Reproducibility

The reproducibility crisis in science stems partly from inadequate infrastructure for capturing and communicating methodological details [40, 2]. SciTeX addresses this through containerized compilation ensuring identical manuscript output across environments [7], figure metadata preserving exact generation parameters [4], execution provenance linking results to code and data [35], and global identifiers enabling cross-module traceability. By embedding reproducibility infrastructure into everyday research tools, SciTeX makes rigorous practices the path of least resistance rather than an additional burden [32, 33].

The platform aligns with emerging frameworks for reproducible research [23, 24, 41] and the five pillars of computational reproducibility [42], while providing practical tools rather than abstract guidelines.

4.5. Limitations

Several limitations warrant acknowledgment:

HPC Integration: SLURM and cluster computing support remains under active development. Current functionality focuses on local and containerized execution.

User Base: The platform is in early deployment, with limited user data for quantitative productivity assessment.

Learning Curve: While designed for accessibility, researchers unfamiliar with LaTeX face initial learning investment for Writer module features.

Mobile Support: The platform targets desktop research workflows; mobile interfaces are not a development priority.

Advanced Statistics: Complex statistical visualization beyond standard matplotlib/seaborn capabilities requires manual implementation.

4.6. Future Directions

Planned development includes automated figure revision using model-driven heuristic correction, integrated citation recommendation based on manuscript content, unified right-panel inspectors across all modules, HPC-native execution pipelines with SLURM integration, automated manuscript consistency checking, and complete provenance graph visualization.

The long-term vision encompasses a research co-pilot capable of running analyses, generating figures, summarizing literature, and producing submission-ready manuscripts—all while maintaining researcher oversight and scientific integrity [12, 38].

5. Conclusions

SciTeX represents both a tool and a philosophy of scientific practice. By combining modular design, reproducibility [3, 1], and AI-native workflows [11], it provides an environment where researchers can think more, fight less with tooling, and create science that stands the test of time.

The platform reduces the activation energy required to conduct rigorous, reproducible science [26]. Each module operates independently while integration across modules provides strictly superior workflows. This architecture—independence with optional integration—reflects realistic adoption patterns where researchers incorporate new tools incrementally.

This manuscript demonstrates that SciTeX can describe itself using its own ecosystem: a self-consistent example of the platform's design goals prepared within the tools it documents.

References

[1] Greg Wilson, Jennifer Bryan, Karen Cranston, Justin Kitzes, Lex Nederbragt, and Tracy K. Teal. Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6):e1005510, 2017. doi: 10.1371/journal.pcbi.1005510.

[2] Lorena A. Barba. Praxis of reproducible computational science. *Computing in Science & Engineering*, 21:73–78, 2018.

[3] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285.

[4] Haim Bar and HaiYing Wang. Reproducible science with LaTeX. *Journal of Data Science*, 19(1):111–125, 2021. doi: 10.6339/21-JDS998.

[5] Christian T. Jacobs. Improving the reproducibility of LaTeX documents by enriching figures with embedded scripts and data. *International Journal of Digital Curation*, 14:292–302, 2020. doi: 10.2218/ijdc.v14i1.656.

[6] Daniel Nüst, Vanessa Sochat, Ben Marwick, Stephen J. Eglen, Tim Head, Tony Hirst, and Benjamin D. Evans. Ten simple rules for writing Dockerfiles for reproducible data science. *PLOS Computational Biology*, 16(11):e1008316, 2020. doi: 10.1371/journal.pcbi.1008316.

[7] Michael Canesche, Roland Leißa, and Fernando Magno Quintão Pereira. Preparing reproducible scientific artifacts using Docker. *arXiv.org*, abs/2308.14122, 2023. doi: 10.48550/arxiv.2308.14122.

[8] Project Jupyter, Matthias Bussonnier, Jessica Forde, Jeremy Freeman, Brian Granger, et al. Binder 2.0—reproducible, interactive, sharable environments for science at scale. *Proceedings of the Python in Science Conference*, pages 113–120, 2018. doi: 10.25080/Majora-4af1f417-011.

[] Daniel S. Himmelstein, Vincent Rubinetti, David R. Slochower, Dongbo Hu, Venkat S. Malladi, Casey S. Greene, and Anthony Gitter. Open collaborative writing with Manubot. *PLOS Computational Biology*, 15 (11):e1007128, 2019. doi: 10.1371/journal.pcbi.1007128.

[9] Albert Krewinkel and Robert Winkler. Formatting open science: Agilely creating multiple document formats for academic manuscripts with Pandoc Scholar. *PeerJ Preprints*, 5:e2648, 2017. doi: 10.7287/peerj.preprints.2648v2.

[10] Markus Konkol, Daniel Nüst, and Laura Goulier. Publishing computational research—a review of infrastructures for reproducible and transparent scholarly communication. *Research Integrity and Peer Review*, 5, 2020. doi: 10.1186/s41073-020-00095-y.

[11] Guiyao Tie, Pan Zhou, and Lichao Sun. A survey of AI scientists. In *arXiv preprint*, 2025.

[12] Ed Li, Junyu Ren, Xintian Pan, Cat Yan, Chuanhao Li, Dirk Bergemann, and Zhuoran Yang. Build your personalized research group: A multiagent framework for continual and interactive science automation. In *arXiv preprint*, 2025.

[13] Karthik Ram. Git can facilitate greater reproducibility and increased transparency in science. *Source Code for Biology and Medicine*, 8:7, 2013. doi: 10.1186/1751-0473-8-7.

[14] Pedro Henrique Pereira Braga, Katherine Hébert, Emma Hudgins, Eric R. Scott, Brandon P. M. Edwards, et al. Not just for programmers: How GitHub can accelerate collaborative and reproducible research in ecology and evolution. *Methods in Ecology and Evolution*, 14:1364–1380, 2023. doi: 10.1111/2041-210X.14108.

[15] Katharine Y. Chen, Maria Toro-Moreno, and Arvind Subramaniam. GitHub is an effective platform for collaborative and reproducible laboratory research. *arXiv.org*, 2024.

[16] Oxford Protein Informatics Group. A match made in heaven: Academic writing with LaTeX and Git. *Blopig Blog*, 2023.

[17] Luka Stanisic, Arnaud Legrand, and Vincent Danjean. An effective Git and Org-Mode based workflow for reproducible research. *ACM SIGOPS Operating Systems Review*, 49:61–70, 2015.

[18] Daniel Nüst, Dirk Eddelbuettel, Dom Bennett, Robrecht Cannoodt, Dave Clark, et al. The Rockerverse: Packages and applications for containerisation with R. *The R Journal*, 12:437, 2020. doi: 10.32614/RJ-2020-007.

[19] Kristina Wiebels and David Moreau. Leveraging containers for reproducible psychological research. *Advances in Methods and Practices in Psychological Science*, 4, 2021. doi: 10.1177/25152459211017853.

[20] Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. ReproZip: Computational reproducibility with ease. *Proceedings of the 2016 International Conference on Management of Data*, pages 2085–2088, 2016. doi: 10.1145/2882903.2899401.

[21] Vicente Giménez-Alventosa, José Damian Segrelles, Germán Moltó, and Miguel Roca-Sogorb. APRICOT: Advanced platform for reproducible infrastructures in the cloud via open tools. *Methods of Information in Medicine*, 59:e33–e45, 2020. doi: 10.1055/s-0040-1712460.

[22] Simone Alessandri, M. Ratto, Sergio Rabellino, et al. CREDO: A friendly customizable, reproducible, Docker file generator for bioinformatics applications. *BMC Bioinformatics*, 25, 2024. doi: 10.1186/s12859-024-05695-9.

[23] Jonathan M. Mang, Hans-Ulrich Prokosch, and Lorenz A. Kapsner. Reproducibility in 2023—an end-to-end template for analysis and manuscript writing. *Studies in Health Technology and Informatics*, 302, 2023. doi: 10.3233/shti230064.

[24] Sabar Dasgupta and Paul Nuyujukian. An open framework for archival,

reproducible, and transparent science. *arXiv.org*, abs/2504.08171, 2025. doi: 10.48550/arxiv.2504.08171.

[25] Yusuke Watanabe. SciTeX: A unified platform for reproducible scientific workflows, 2025. URL `https://scitex.ai`. Modular framework integrating manuscript preparation, visualization, literature management, and computational environments.

[26] The Turing Way Community. *The Turing Way: A Handbook for Reproducible, Ethical and Collaborative Research*. Zenodo, 2022. doi: 10.5281/zenodo.3233853.

[27] Peter Kastrup. Tectonic: A modernized, self-contained LaTeX engine, 2024. URL `https://tectonic-typesetting.github.io/`.

[28] Cheng Xu and contributors. GitHub Action for LaTeX build automation, 2024. URL `https://github.com/xu-cheng/latex-action`.

[29] Armin Ariamajd, Raquel López-Ríos de Castro, and Andrea Volkamer. PyPackIT: Automated research software engineering for scientific Python applications on GitHub. *arXiv.org*, abs/2503.04921, 2025. doi: 10.48550/arxiv.2503.04921.

[30] Stephen R. Piccolo, Zachary E. Ence, Elizabeth C. Anderson, Jeffrey T. Chang, and Andrea H. Bild. Simplifying the development of portable, scalable, and reproducible workflows. *eLife*, 10, 2021. doi: 10.7554/eLife.71069.

[31] Meghan A. Balk, John Bradley, M. Maruf, B. Altıntaş, Yasin Bakış, et al. A FAIR and modular image-based workflow for knowledge discovery in the emerging field of imageomics. *Methods in Ecology and Evolution*, 15:1129–1145, 2024. doi: 10.1111/2041-210X.14327.

[32] Ben Marwick, Carl Boettiger, and Lincoln Mullen. Packaging data analytical work reproducibly using R (and friends). *The American Statistician*, 72(1):80–88, 2018. doi: 10.1080/00031305.2017.1375986.

[33] Aaron Peikert, Caspar V. van Lissa, and Andreas M. Brandmaier. Reproducible research in R: A tutorial on how to do the same thing more than once. *Psych*, 2021. doi: 10.31234/osf.io/fwxs4.

[34] Nikolas I. Krieger, Adam Perzynski, and Jarrod Dalton. Facilitating reproducible project management and manuscript development in team science: The projects R package. *PLoS ONE*, 14, 2019. doi: 10.1371/journal.pone.0212390.

[35] Christian Schulz. Reproducible data citations for computational research. *arXiv (Cornell University)*, abs/1808.07541, 2022. doi: 10.48550/arxiv.1808.07541.

[36] Juan Pedro Araque Espinosa et al. A continuous integration and web framework in support of the ATLAS publication process. *Journal of Instrumentation*, 16, 2020. doi: 10.1088/1748-0221/16/05/T05006.

[37] Karl-Ludwig Besser. Review response template. `https://www.overleaf.com/latex/templates/review-response-template/tmbvmjstxwrd`, 2025.

[38] OpenLens Team. OpenLens AI: Fully autonomous research agent for health informatics. *arXiv preprint arXiv:2509.14778*, 2025.

[39] Carl Boettiger. rrtools: Tools for writing reproducible research in R. `https://github.com/benmarwick/rrtools`, 2019.

[40] Lorena A. Barba. Terminologies for reproducible research. *arXiv preprint arXiv:1802.03311*, 2018. doi: 10.48550/arXiv.1802.03311.

[41] Lázaro Costa, Susana Barbosa, and Jácome Cunha. A framework for supporting the reproducibility of computational experiments in multiple scientific domains. *Future Generations Computer Systems*, abs/2503.07080, 2025. doi: 10.48550/arxiv.2503.07080.

[42] Ashley M. Zehr et al. The five pillars of computational reproducibility: Bioinformatics and beyond. *Genome Biology*, 24(1):1–15, 2023. doi: 10.1186/s13059-023-03087-0.

6. Resource Availability

6.1. Lead Contact

Further information and requests for resources should be directed to and will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

6.2. Materials Availability

This study did not generate new unique reagents.

6.3. Data and Code Availability

- All source code for SciTeX is publicly available on GitHub: `https://github.com/scitex-ai/scitex`
- The SciTeX Python package is available via PyPI: `pip install scitex`
- Documentation is available at: `https://docs.scitex.ai`
- The cloud platform is accessible at: `https://scitex.ai`
- This manuscript's source files are included in the SciTeX Writer repository as a demonstration
- DOI for archived version: [Zenodo DOI to be assigned upon submission]

All code is released under the GNU General Public License v3.0 (GPL-3.0), ensuring open access and modification rights. The platform follows FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data and code artifacts.

Any additional information required to reanalyze the data reported in this paper is available from the lead contact upon request.

Ethics Declarations

All study participants provided their written informed consent ...

Author Contributions

Y.W., T.Y., and D.G. conceptualized the study ...

Acknowledgments

This research was funded by funding bodies here

Declaration of Interests

The authors declare that they have no competing interests.

Declaration of Generative AI in Scientific Writing

The authors employed large language models such as Claude (Anthropic Inc.) for code development and complementing manuscript's English language quality. After incorporating suggested improvements, the authors meticulously revised the content. Ultimate responsibility for the final content of this publication rests entirely with the authors.

Tables

Option	Description	Example
–engine ENGINE	Force specific compilation engine	–engine tectonic
–draft	Draft mode (single-pass compilation)	–draft
–no-figs	Skip figure processing	–no-figs
–no-tables	Skip table processing	–no-tables
–no-diff	Skip difference generation	–no-diff
–watch	Enable hot-recompile file watching	–watch
–clean	Clean build (remove all cache)	–clean
–verbose	Verbose compilation output	–verbose

Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ∼15s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./.cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.

Figures

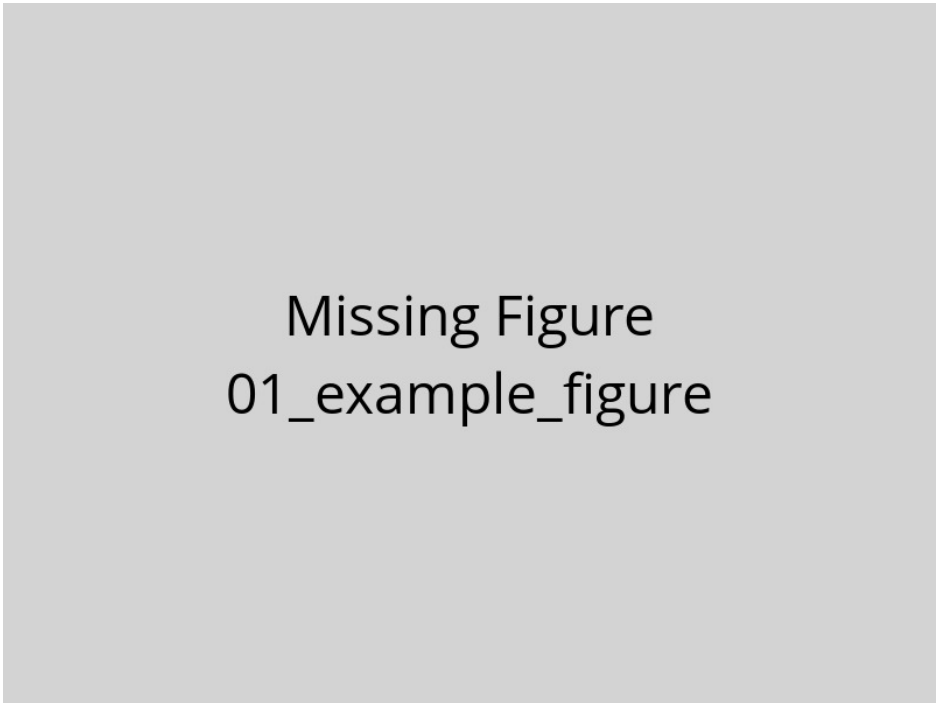

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.

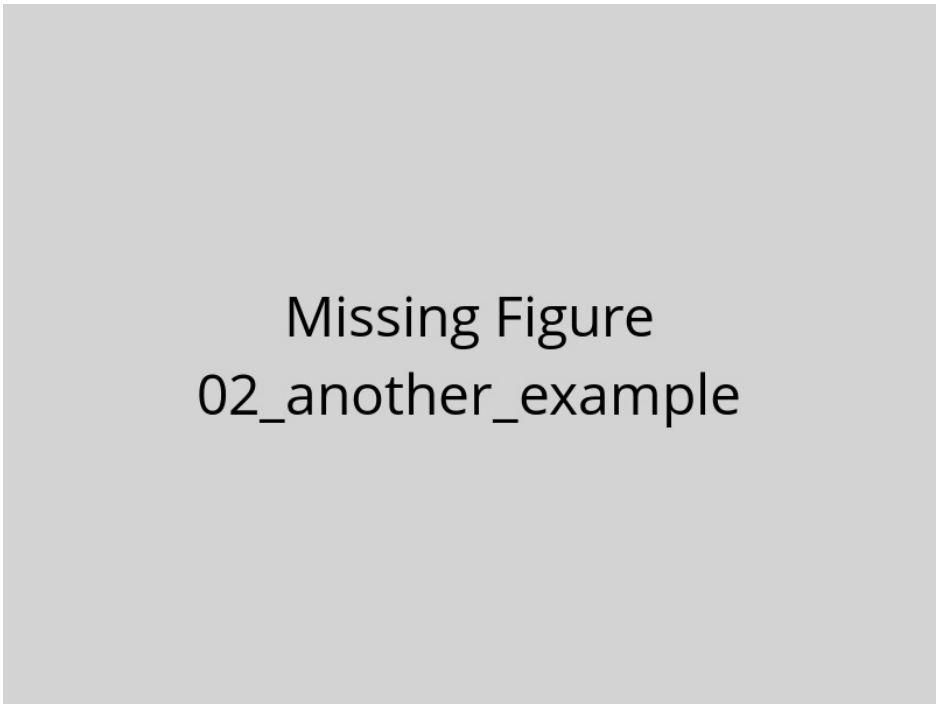

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

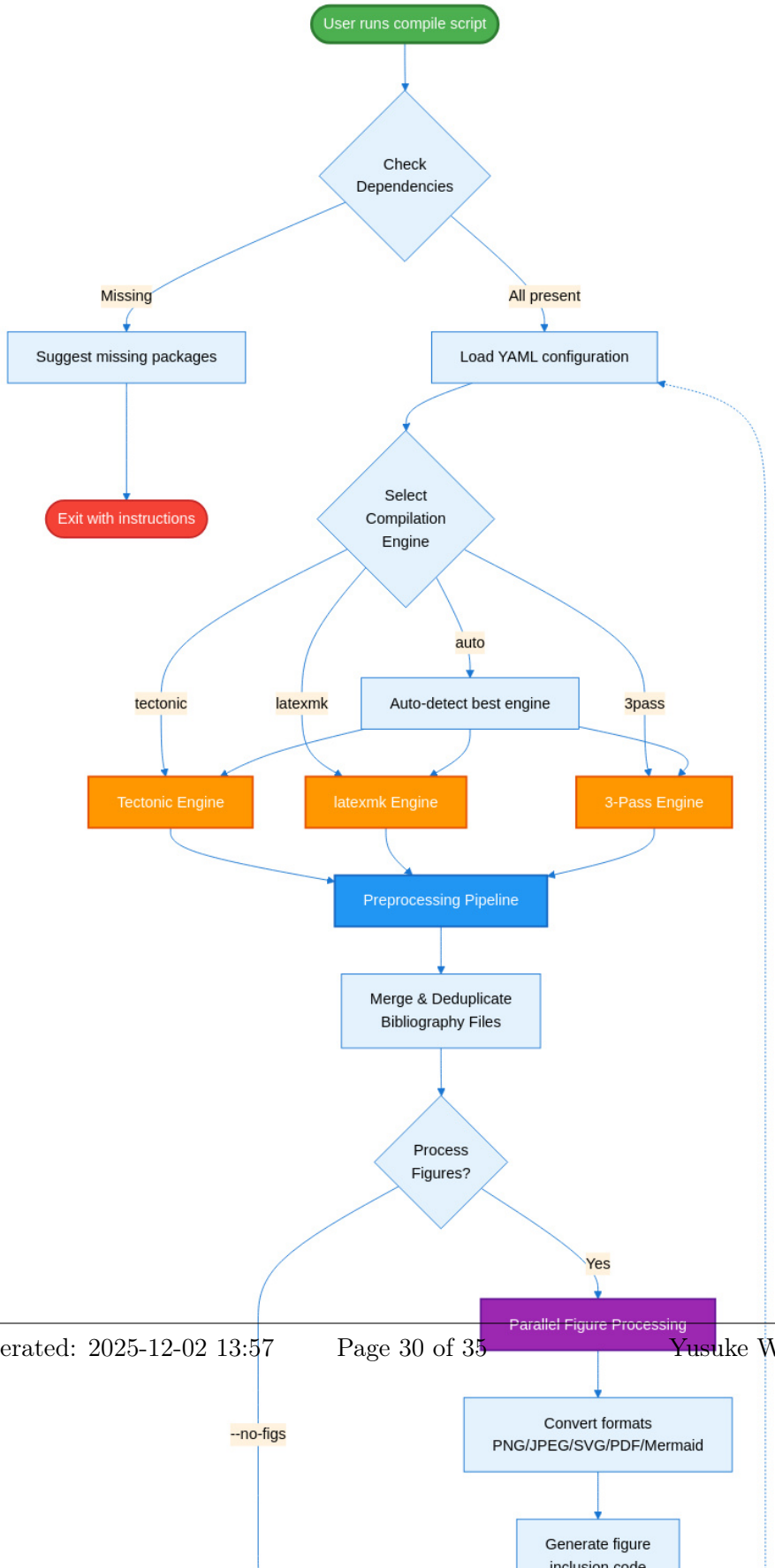
User runs compile script
Check Dependencies
Missing
All present
Suggest missing packages
Load YAML configuration
Exit with instructions
Select Compilation Engine
auto
tectonic
latexmk
Auto-detect best engine
3pass
Tectonic Engine
latexmk Engine
3-Pass Engine
Preprocessing Pipeline
Merge & Deduplicate Bibliography Files
Process Figures?
Yes
Parallel Figure Processing
--no-figs
Convert formats PNG/JPEG/SVG/PDF/Mermaid
Generate figure

Figure 4 – SciTeX Writer Directory Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

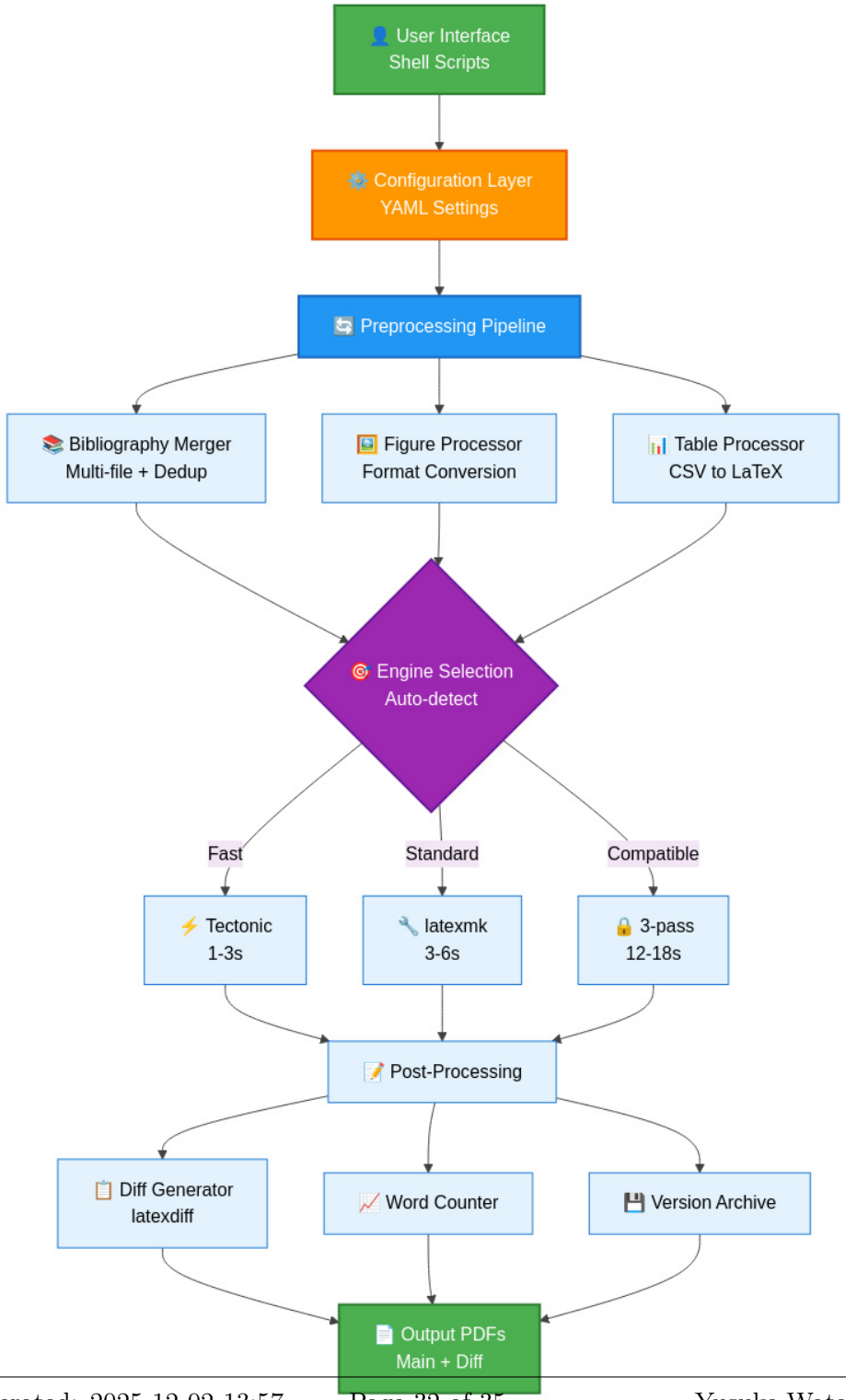

Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

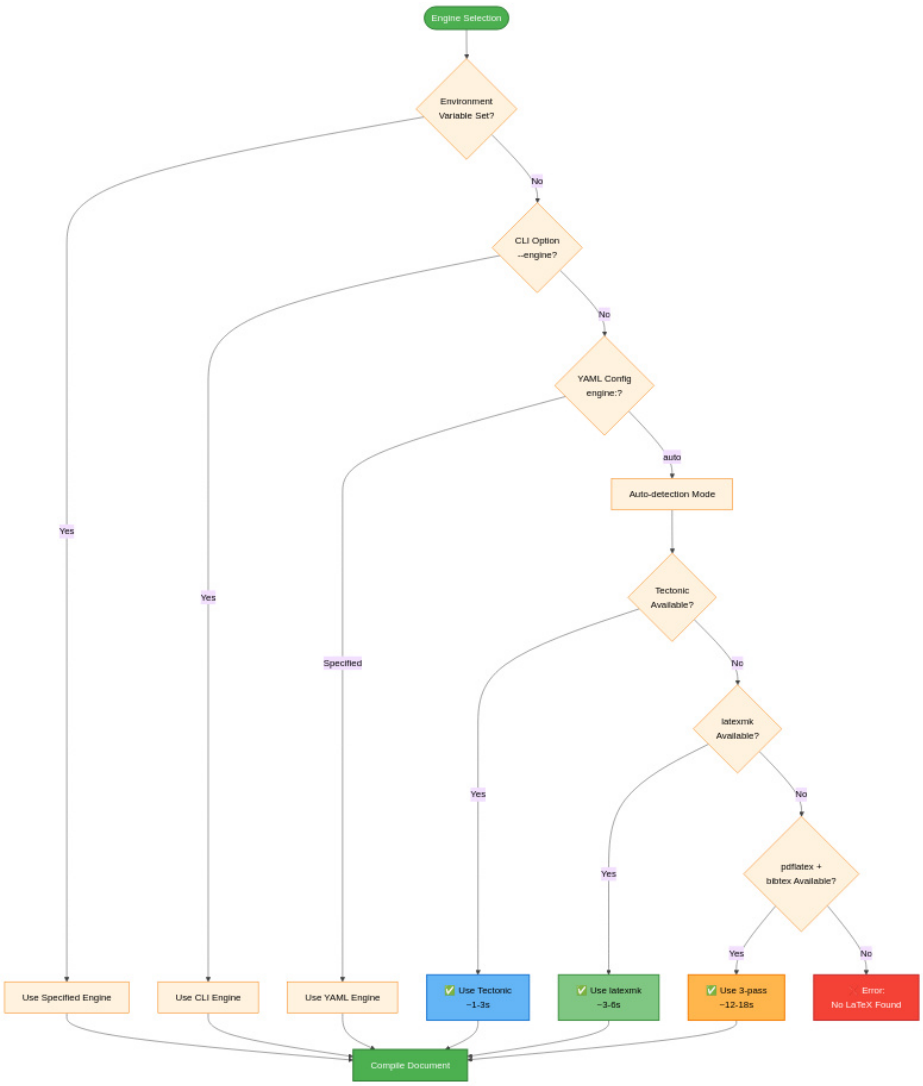

Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (–engine) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build durations on reference hardware (16 GB RAM, 8 cores).

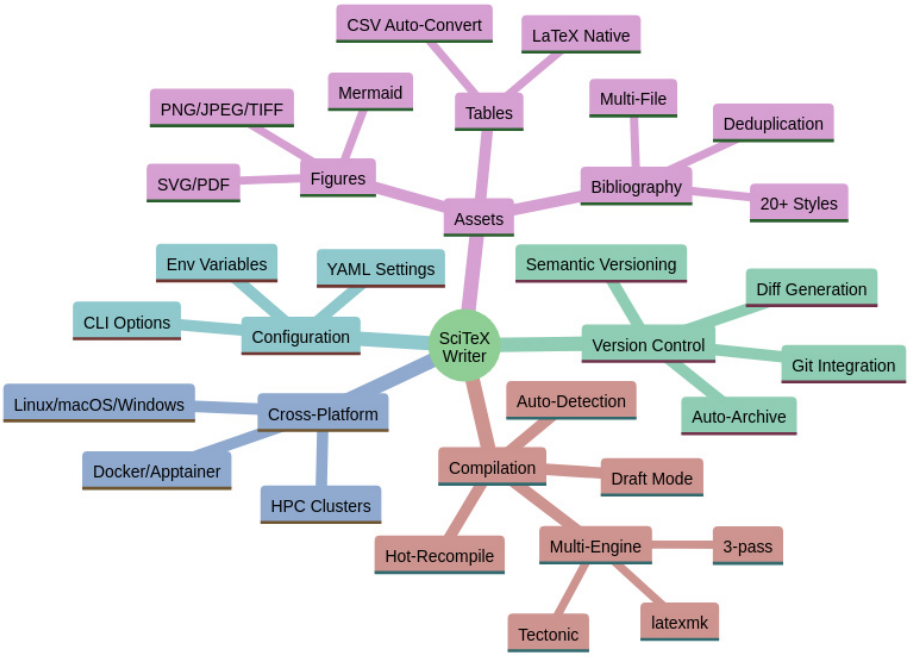

Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework's major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.

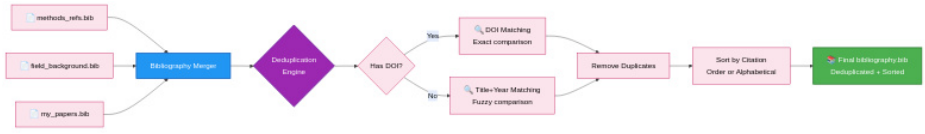

Figure 8 – Multi-File Bibliography Processing and Deduplication. The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).